Lecture 2

Process and Process APIs

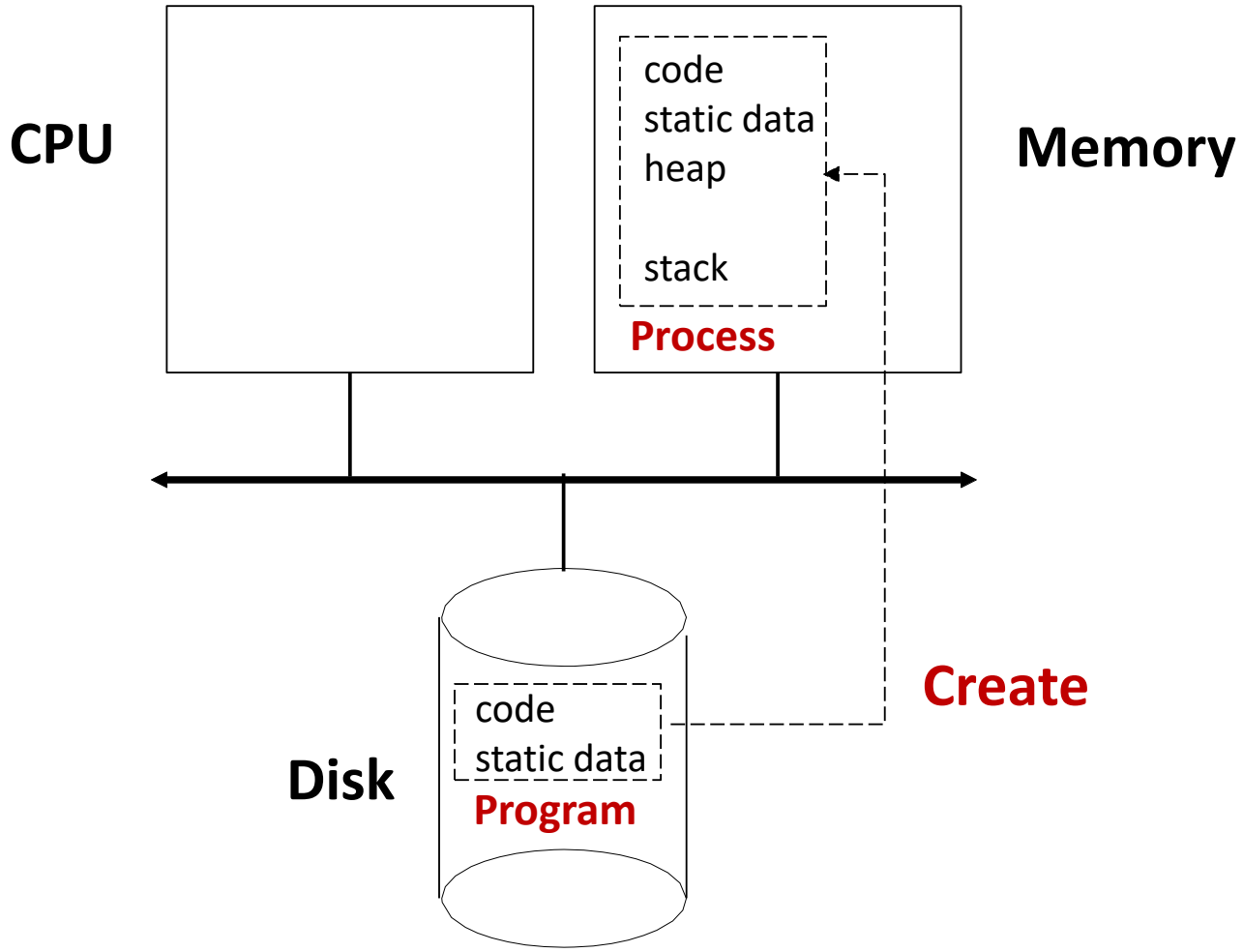
Lecture 2

Process and Process APIs

Previously …

- What's OS?
- Learning in 3 modules: Virtualization, Concurrency, and Persistence
- Virtualization
  - Processor (2) and memory (3) virtualization
- Concurrency
  - Concurrency (4) of the many resource virtualizations
- Persistence
  - I/O device (2)

From Program to Process

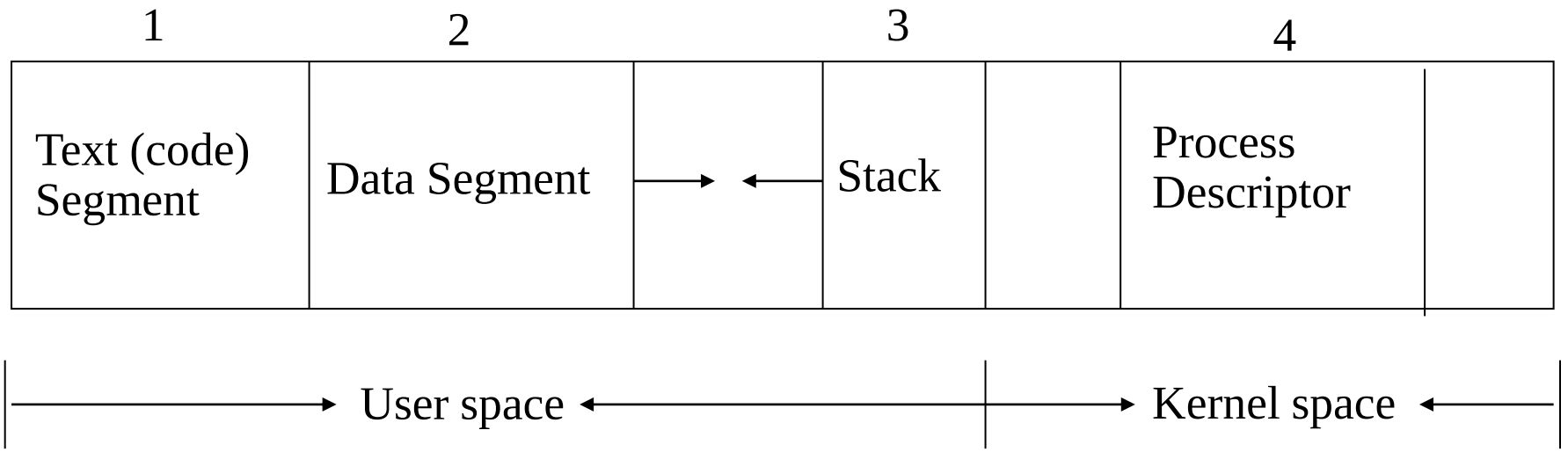


What is a Process?

- A Process is an entity with 4 components:
  Process = (P, D, C, S)
  - P is a program or a piece of code that the process is associated with
  - D is a processing data, defined by a set of data variables that hold user data
  - C is a control state, defined by a set of control variables that indicate where to execute next
  - S is a process state – ready, suspended, running, etc.

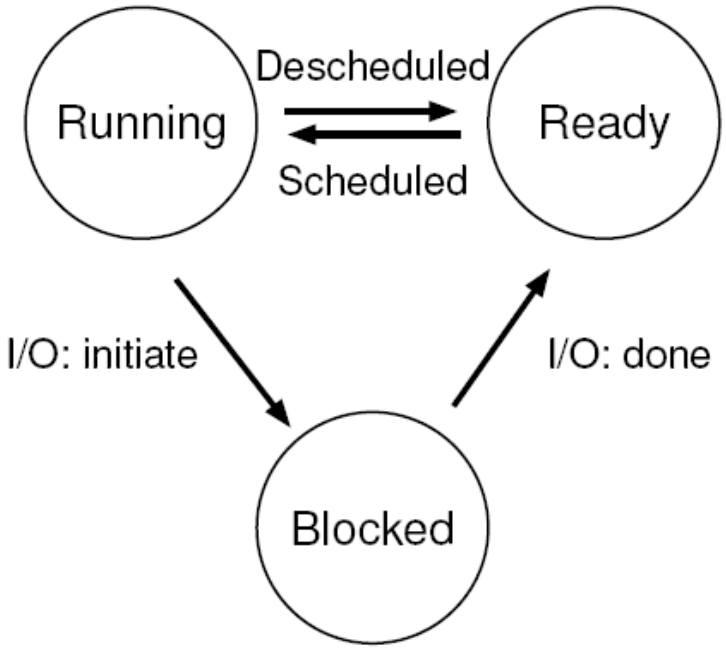
A Process Space

- A memory that the process uses to reference its instructions and data
  1. Code Segment containing instructions
  2. Data Segment containing heap data (malloc)
  3. Stack containing parameters, local variables, return address, etc. of functions
  4. Process Descriptor (keep Process Control Block) – stored in the kernel space and can only be accessed in kernel mode execution

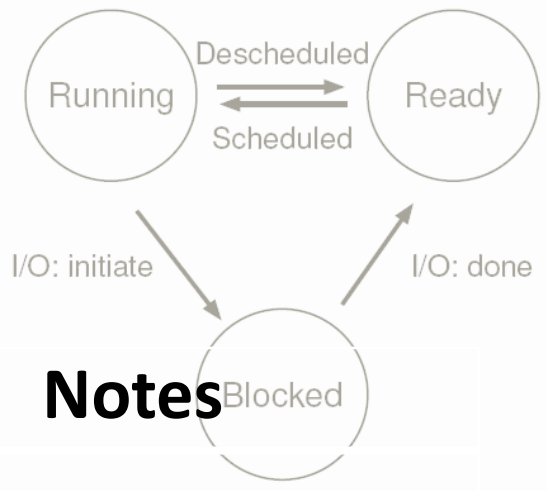


Process State

- Ready: a process is ready to run
- Running: a process is running on a processor
- Blocked: a process has performed some operation that means it not ready to run, it is then suspended

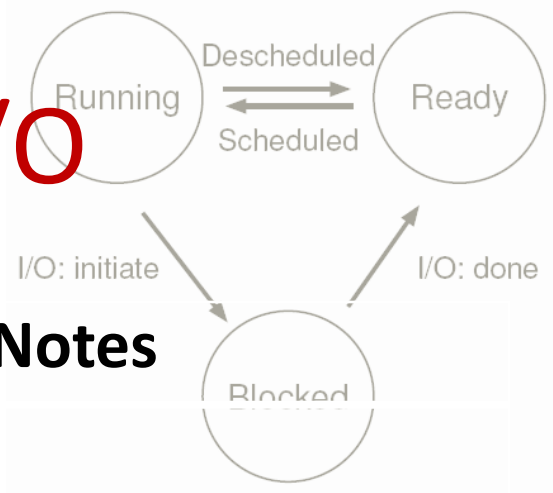


Process State – CPU only



Time	Process$_0$	Process$_1$	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	
4	Running	Ready	Process$_0$ now done
5	–	Running	
6	–	Running	
7	–	Running	
8	–	Running	Process$_1$ now done

Process State – CPU and I/O



Time	Process$_0$	Process$_1$	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	Process$_0$ initiates I/O
4	Blocked	Running	Process$_0$ is blocked,
5	Blocked	Running	so Process$_1$ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	Process$_1$ now done
9	Running	–	
10	Running	–	Process$_0$ now done

Typical Process APIs

- Create: when double-click an application icon, an OS routine is invoked to create a process to run the program
- Destroy: user may wish to kill a process
- Wait: sometimes useful to wait for a process to stop running
- Misc control: e.g. suspend or resume a process
- State: get status information (how long a process has run for, what state it is in)

OS Data Structure

- The OS is a program, and like any program, it has some key data structures that track various relevant pieces of information
- The OS will
  - keep process lists for all processes that are ready and running
  - keep track of blocked processes – when the I/O event completes, wake up the right process
  - keep track of register context (registers' states) and save it to memory if a process is blocked
- The individual structure that stores information about a process is called the Process Control Block (PCB)

Process Control Block (PCB)

- The Process Control Block stores information associated with each process:
  - Process identity
  - Process state
  - Program counter
  - CPU registers
  - CPU scheduling information
  - Memory-management information
  - Accounting information
  - I/O status information

Process and OS

- Process is a running program, there are many and many of them running when a machine is turned on
- CPU virtualization
  - The OS creates an illusion that there is an infinite supply of CPUs for processes
  - Multiprogramming/Time-sharing –> taking turns to use the CPU
- To do this, the OS needs:
  - Low-level **mechanisms** – a context switch (to stop/pause and start processes)
  - High-level **scheduling policies** – algorithms for making decisions
    - e.g. with many programs to run, who goes first and for how long?
  - Mechanisms and policies should be separated

MECHANISM

Challenges of Virtualization

- Attaining performance while maintaining control is one of the central challenges in building an operating system
  - Performance: how can we implement virtualization without adding excessive overhead
  - Control: how can we run processes efficiently while retaining control over the CPU

Direct Execution Mechanism

OS	Program
Create entry for process list	
Allocate memory for program	
Load program into memory	
Set up stack with argc/argv	
Clear registers	
Execute **call** main()	
	Run main()
	Execute **return** from main
Free memory of process	
Remove from process list	

Sample Problems

- If we just run a program, how can the OS make sure the program doesn't do things that we don't want, while still running it efficiently?

- When a process is running, how does the operating system stop it from running and switch to another process.

Suggested Mechanism

- Limited Direct Execution Mechanism
  1) Restricted Operation
  2) Switch Between Processes
  3) Timer Interrupt

Limited Direct Execution

OS @ boot (kernel mode)	Hardware	
initialize trap table		
	remember address of...	
	syscall handler	

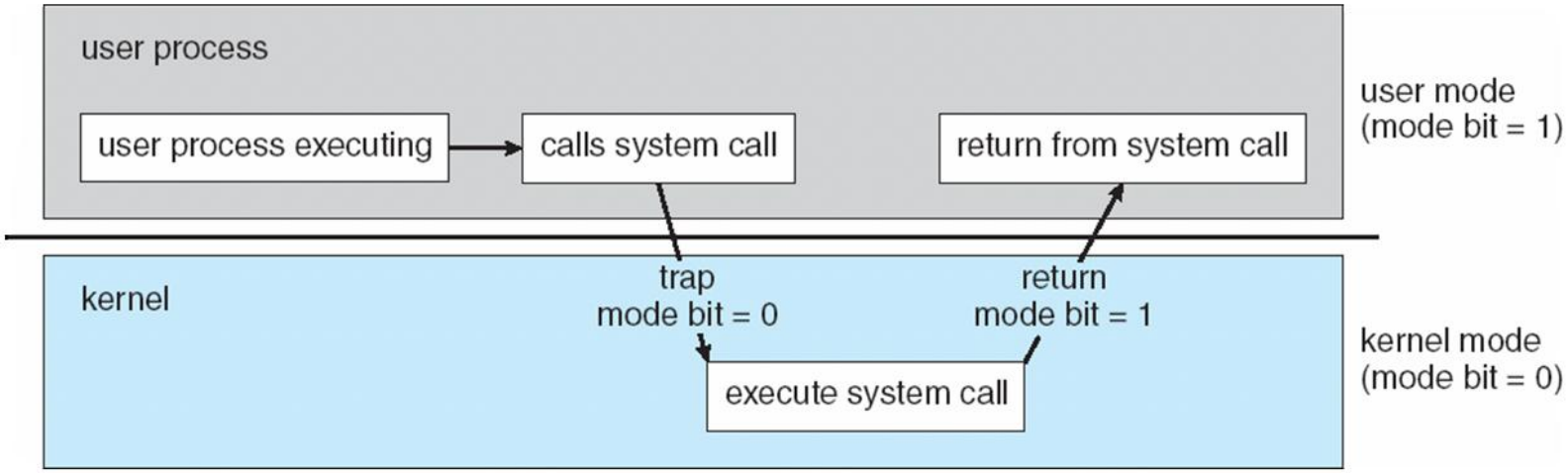
OS @ run (kernel mode)	Hardware	Program (user mode)
Create entry for process list		
Allocate memory for program		
Load program into memory		
Setup user stack with argv		
Fill kernel stack with reg/PC		
return-from-trap		
	restore regs from kernel stack	
	move to user mode	
	jump to main	
		Run main()
		...
		Call system call
		trap into OS
	save regs to kernel stack	
	move to kernel mode	
	jump to trap handler	
Handle trap		
Do work of syscall		
return-from-trap		
	restore regs from kernel stack	
	move to user mode	
	jump to PC after trap	
		...
		return from main
		trap (via `exit()`)
Free memory of process		
Remove from process list		

1) Restricted Operation

- Kernel mode: code that runs can do what it likes
- User mode: any code that runs in user mode is restricted in what it can do. EX Can't request I/O
- EX: If a user process wants to access disk, hardware provides the ability for user programs to perform a system call.
  - To execute a system call, a program must execute a special instruction to jumps into the kernel, called trap
  - Raises the privilege level to kernel mode
  - Perform privileged operations
  - When finished, the OS calls a return-from-trap instruction to return into the calling user program and reduce the privilege level back.

User Mode and Kernel Mode

- Mode bit provided by hardware
  - Provides ability to distinguish whether system is running user code or kernel code
  - Privileged instructions only executable in kernel mode
  - System call changes mode to kernel, return resets it to user



System Call

A system call, such as fopen() or read(), looks like a typical function call in C. How does the system know it's a system call?

- When you call fopen(), you are executing a function call into the C library.
- The library uses an agreed-upon calling convention with the kernel, which include calling the trap instruction
- Note that the parts of the C library that make system calls are hand-coded in assembly

How does the trap know which code to run inside the OS ?

- OS tells the hardware what code to run when certain exceptional events occur through the trap table.
- For example, what code should run when a hard-disk interrupt takes place
- The kernel does so by setting up a trap table at boot time

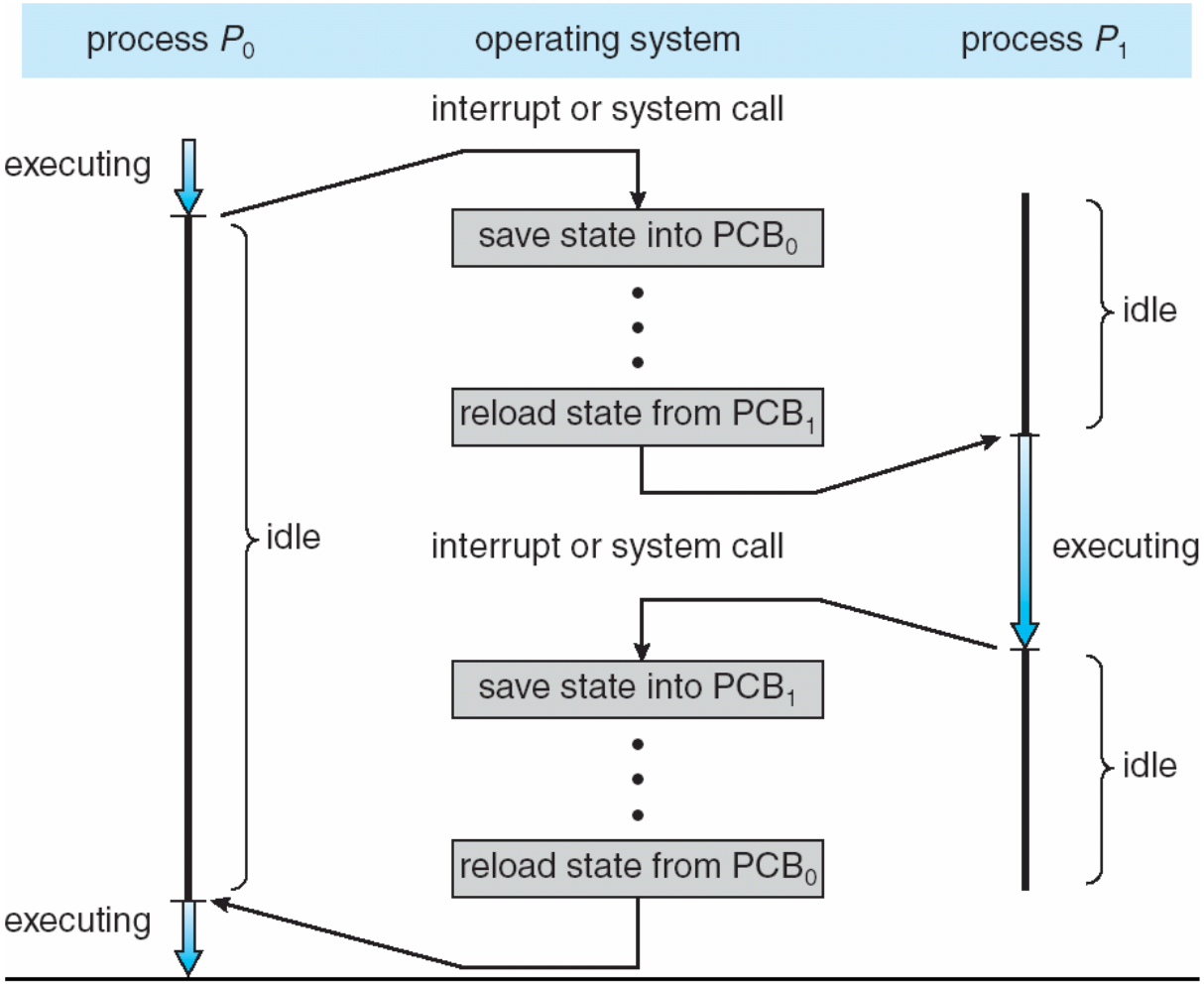
what types of horrible things could you do to a system if you could install your own trap table?

2) The Switch Between The Processes

- This switch decision is made by a part of the OS known as the scheduler.
- If the decision is made to switch, the OS then executes a low-level piece of code which we refer to as the context switch.

Context switch is the step of storing and restoring the state of a process so that execution can be resumed from the same point at a later time.

Saving and Restoring Context



Context Switch

- Saving and Restoring Context
  - OS saves the current context by executing some low-level assembly code to save the general purpose registers, PC, as well as the kernel stack pointer. (save PCB)
  - Then restore the PCB context of the soon-to-be-executing process. New processes are chosen from a ‘ready’ queue
  - When the OS executes a return-from-trap instruction, context switch is completed

Cooperative Level with OS

- Challenge
  - if a process is running on the CPU, this means the OS is not running.
  - If the OS is not running, how can it do anything at all ?
- How can the OS regain control of the CPU so that it can switch between processes?
  - A cooperative approach: Wait for system calls
  - A non-cooperative approach: The OS takes control

A Cooperative Approach (1)

- OS trusts the users' processes to behave reasonably
- Processes that run for too long are assumed to periodically give in the CPU for the OS, so it can run some other process
- There is an explicit yield system call, which does nothing except to transfer control to the OS
- Most processes transfer control of the CPU to the OS quite frequently by making system calls or performing illegal instructions
  - Calls to open a file and subsequently read it
  - Calls to send a message to another machine
  - Calls to create a new process
  - Calls to yield CPU
  - Divide by zero
  - Access memory illegally

A Cooperative Approach (2)

- In a cooperative system, the OS regains control of the CPU by waiting for a system call or an illegal operation of some kind to take place
- What happens, if a process ends up in an infinite loop (buggy), and never makes a system call?
- What can the OS do?
  - Nothing really, must reboot the machine

A Non-cooperative Approach

- A timer interrupt can be used (a timer device, programmed to raise an interrupt periodically)
- When the interrupt is raised:
  - the currently running process is halted
  - a preconfigured interrupt handler in the OS runs
  - OS has regained control of the CPU, and can do what it pleases, e.g. stop the current process
- Note that
  - Hardware must save enough states of the running program
  - A subsequent return-from-trap instruction should be able to resume
  - Similar to explicit system-call trap

(3) Timer Interrupt

OS @ boot (kernel mode)	Hardware
initialize trap table	
	remember addresses of...
	syscall handler
	timer handler
start interrupt timer	
	start timer
	interrupt CPU in X ms

OS @ run (kernel mode)	Hardware	Program (user mode)
		Process A
		...
	timer interrupt	
	save regs(A) to k-stack(A)	
	move to kernel mode	
	jump to trap handler	
Handle the trap		
Call `switch()` routine		
save regs(A) to proc-struct(A)		
restore regs(B) from proc-struct(B)		
switch to k-stack(B)		
return-from-trap (into B)		
	restore regs(B) from k-stack(B)	
	move to user mode	
	jump to B's PC	
		Process B
		...

Heart of LDE Mechanism

1) Restricted Operation (Execution Mode)
2) Switch Between Processes (Context Switch)
3) Timer Interrupt (Hardware Support)

Guess how long does something like a context switch take?

- sub-microsecond on systems with 2- or 3-GHz processors

Summary

- The OS, first (during boot time), sets up the trap handlers and starts an interrupt timer
- Then runs processes in a restricted operation
- The OS can feel quite assured that processes can run efficiently, only requiring OS intervention to perform privileged operations
- When a process monopolizes the CPU for too long, it is switched out on an interrupt
- This is the basic mechanism for virtualizing the CPU
- Question: Which process should be executed at a given time?
  - Answer: The scheduler with rules and protocols will decide

Some important system calls

PROCESS APIs OF UNIX

fork(): Hello World

The **fork()** system call is used to create a new **process**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int
main(int argc, char *argv[])
{
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {
        // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {
        // parent goes down this path (original process)
        printf("hello, I am parent of %d (pid:%d)\n",
                rc, (int) getpid());
    }
    return 0;
}
```

fork(): Execution Result

- When you run this program, what you see is the following:

  hello world (pid:29146)

  hello, I am parent of 29147 (pid:29146)

  hello, I am child (pid:29147)
- Look at the first line under main()
  - The process prints out a hello world message, with its process identifier, also known as a PID
  - In Unix systems, the PID is used to name the process
- Note that the process created by **fork()** is an (almost) exact copy of the calling process
- However, the new process has its own address space (i.e. its own private memory, its own registers, its own PC, etc.)
- So…OS will see there are now two processes in the system

fork(): Some Notes

- The newly-created process is called the child, the creating process is called the parent
- The child doesn't start running at main(); rather, it just comes into life as if it had called **fork()** itself
- If a computer has a single CPU, either the child or the parent might run at any particular point in time
- Notice that
  - The first "hello world" just got printed once.
  - The output is not deterministic, i.e. different executions may give output in a different order

wait(): Example Usage

- Sometimes, it is quite useful for a parent to wait for a child process to finish before moving on
- Using **wait()** or **waitpid()** will force the parent to wait
- Let's modify the last printf() of the previous code:

```
// parent goes down this path (original process)
{
        int  wc = wait(NULL);
        printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
                rc, wc, (int) getpid());
}
```

- The parent calls **wait()** to delay its execution until the child finishes executing – when the child is done, **wait()** returns to the parent

wait(): Result and Notes

- With this code, we now know that the child will always print first!

  hello world (pid:29266)

  hello, I am child (pid:29267)

  hello, I am parent of 29267 (wc:29267) (pid:29266)

- More generally:
  - **wait(int ****status***)** suspends execution of the current process until one of its children terminates
  - **waitpid(pid** *pid***, int ****status***, int** *options***)** suspends execution of the current process until specified child terminates
  - Both return PID as result and exit status in *status* argument

exec(): Family of Functions

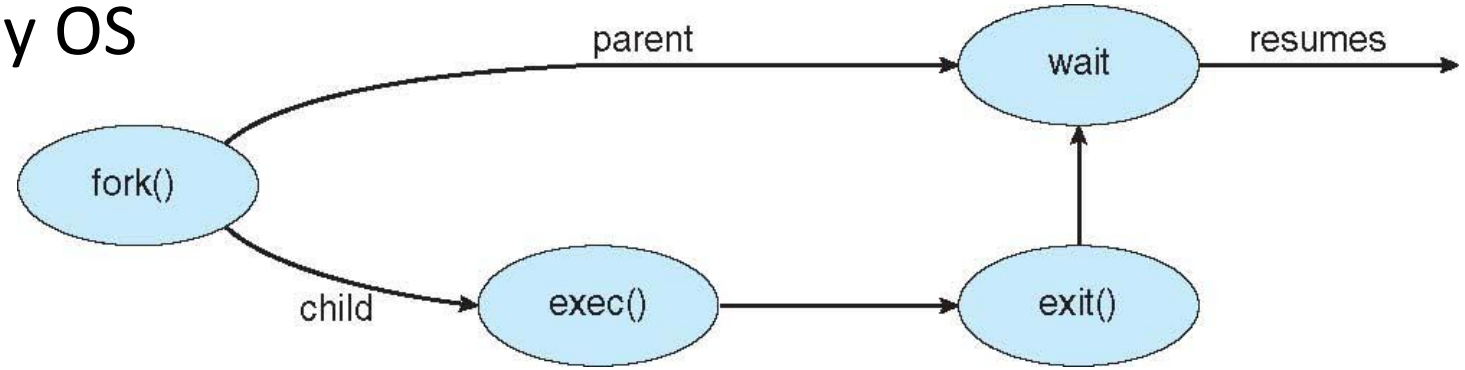
- There is a family of **exec()** system calls, e.g. **execvp()**
- These are useful when you want to run a child program that is different from the parent program
- What it does: given the name of an executable (e.g. word_count), and some arguments (e.g. p.txt)

```
// child (new process)
char *myargs[3];
myargs[0] = strdup("word_count"); // program: "word_count”
myargs[1] = strdup(“p.txt"); // argument: file to count
myargs[2] = NULL; // marks end of array
execvp(myargs[0], myargs); // runs the word count code
printf("this shouldn’t print out");
```

- A successful call to an **exec()** function never returns

Termination

- A process terminates its execution by:
  - making an exit system call, e.g. **exit(***status***)**
  - or abnormally due to a fatal error or signal
- The exit status can be retrieved by the parent process via the **wait()** system call
- On termination, the process resources are deallocated by OS



Other APIs

- **kill()**: send signal to a process, including directives to go to sleep, die, etc.
- Signals subsystem
  - delivers external events to processes
  - Provides ways for processes to receive and process those signals
- Command line tools
  - ‘ps’ – lists the running processes
  - ‘top’ - displays the processes and how much CPU and other resources they are using

Why fork() and exec()? (1)

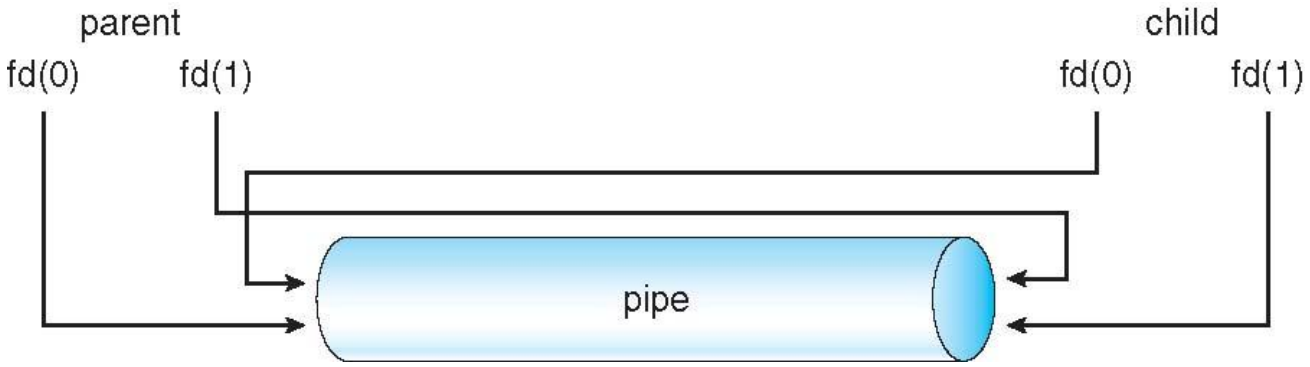
- The separation of **fork()** and **exec()** is essential in building a Unix shell (command-line-interpreter) because it lets the shell run code after the call to **fork()** but before the call to **exec()**
- With the name of an executable program, plus any arguments passed to the Unix shell, it:
  - figures out where the executable is
  - calls **fork()** to create a new child process
  - alters the environment of the about-to-be-run program, if necessary
  - calls some variant of **exec()** to run the command
  - waits for the command to complete by calling **wait()**
  - when the child completes, the shell returns from the **wait()** and prints out a prompt again, ready for the next command

Why fork() and exec()? (2)

- The separation of **fork()** and **exec()** allows the shell to do many useful things rather easily
  - Suppose the command is: prompt> wc p.txt > newfile.txt
  - Before calling **exec(),** the shell redirects the output by closing the standard output and opening the file newfile.txt
- Unix pipes are implemented in a similar way but with the **pipe()** system call
  - The output of one process is connected to the write-end of the pipe, and the input of another process is connected to the read-end of that same pipe

Pipes

- Pipes allow communication in a producer/consumer style
  - Producer writes to one end (the write-end of the pipe)
  - Consumer reads from the other end (the read-end of the pipe)
  - If the parent wants to receive data from the child, it should close *fd[1]* and read from *fd[0]*, and the child should close *fd[0]* and write to *fd[1]*



  - **int pipe(int** *fd[2]***);** *fd[0]* is set up for reading, *fd[1]* for writing